WELLPATH

Role-Based Lifestyle Analytics and Health Monitoring Platform

Final Project Report

Course: CST3117 Project II

Team members: Mahir Mazumder, John Junior Omage, Buntong Ngy, Yeven Yao, Yonathan Genetu

Instructor: Kunal Parikh

Submission date: 21 August 2026

Table of Contents

Executive Summary

WellPath is a health and wellness prototype built around a simple problem. People collect a lot of health information, but it often sits in different apps and systems. It can be hard to understand what it means when it is spread out. Across Project I and Project II, we explored how clinical measurements, wearable data, and daily habits could be shown in one place. The goal was to help patients and the people supporting them see useful changes over time. From the start, WellPath was meant to support wellness tracking. It was not meant to diagnose a medical condition.

The original Project I proposal [1] focused on a shared platform for patients, clinicians, and trainers. Each user type would have its own view. The system would show trends, compare more than one health factor, and give practical lifestyle guidance. Project II expanded that idea. We added more patient features, new scores and KPI views, AI explanations, mood and nutrition tracking, cycle tracking, stronger clinician and trainer tools, an admin area, and a separate machine learning research project. Most of these features were added after feedback and after we reviewed the client discussion again. We also felt that a basic dashboard did not show enough of the idea.

The project did not follow a simple path. A major team issue forced us to change the schedule and move work between team members. More integration work also had to happen near the end of the semester. Client contact was stronger early in the project and became limited later. This left us with less clear outside guidance while the scope was changing. We also could not find a clinician who could review the final health content with us. Because of this, we used public research data, kept our wording careful, and did not claim that the system was medically validated.

Even with those limits, the final result is a working prototype. It has separate spaces for each user role, stored patient data, trend and score views, optional AI explanations, and research-based risk signals. The research also tested one of our early assumptions. We thought that adding more lifestyle and background information would always improve health predictions. The results did not fully support that idea. Some factors helped and others did not. That was useful because it showed us where the concept was stronger and where it still needed work. WellPath was successful as an academic prototype, but

it would still need medical review, real user data, formal testing, and more client input before real use.

1. Introduction

1.1 Motivation for the Project

WellPath started with a gap we saw in current health tools. People now collect information about activity, sleep, heart rate, food, mood, and clinical measurements. Much of it is kept in separate places. One result on its own may not say much. A short medical appointment also gives only a small view of what happened between visits. Our Project I proposal asked whether these pieces could be brought together into one clearer view over time.

Our first discussion with Saman helped set the direction. We were not given a fixed feature list. The discussion focused on a wider problem with disconnected health information and missing lifestyle context. This gave us room to explore recommendations, prediction models, and different views for patients and professionals. It also meant that the team had to make many choices about what was useful and realistic for a student project.

1.2 Background Information

Project I described WellPath as a health and lifestyle platform that would not provide a diagnosis. The main users were patients, clinicians, and trainers. The goals were to bring separate health records together, show changes over time, point out patterns that may need attention, and explain the information in plain language. The first design used sample data such as blood pressure, BMI, resting heart rate, steps, sleep, food, stress, exercise, and sitting time. We were not trying to prove that one factor caused a health outcome. We wanted to see how the information could be organized and reviewed together.

The problem also looked different depending on the user. A patient needed a quick view that could be checked without learning analytics terms. A trainer needed activity, recovery, plans, and feedback for the people assigned to them. A clinician needed longer histories, notes, signals, and follow-up work. An administrator needed account

and access information without opening private health records. This difference became the reason for using separate role workspaces instead of copying one dashboard four times.

Project I planned to use made-up data for the application and public data for research [1]. That choice reduced privacy risk and allowed the team to develop without collecting health records from classmates or outside users. It also created an important limit. A screen could demonstrate a workflow, but it could not prove that the workflow would improve care. We kept that difference in mind when describing the final result.

1.3 Goals and Scope

Project II aimed to turn that idea into a working prototype without losing the main purpose. We wanted separate experiences for each user role, stored data instead of static screens, and trend and score views that were easy to understand. We also wanted to test whether public health data could support useful risk signals without making a diagnosis. Later in the semester, we added an admin role and gave the patient, trainer, and clinician areas more detail.

We kept several limits in place throughout the project. WellPath was not built to diagnose a condition, prescribe treatment, replace a healthcare professional, or monitor an emergency. We did not collect real patient records or run a clinical study. The app used made-up demonstration data, while the research used public datasets. We also did not have a medical professional approve the scores or health explanations. For that reason, the scores, recommendations, risk signals, and AI text were presented as wellness information only.

The original proposal listed five main goals. It aimed to combine several health measures, provide different views by role, look at more than one factor at a time, give practical guidance, and show changes over time [1]. Project II used those goals as a guide, but the work did not move forward evenly. Trend views and role separation became working features early. Wider social factors, tested models, and clearer statements about uncertainty came later.

For this report, a goal is treated as met only when it appears in the working demonstration and can be followed through a complete user action. A designed screen by itself is not counted as full completion. This is why the report separates implemented workflows from planned connections such as Apple Health, Health Connect, stored survey answers, and direct use of the research pipeline.

1.4 Overview of the Implementation Process

The implementation changed often. We started Project II with the Project I interface, an early database plan, proposed KPI groups, and a schedule. Our first tasks were to clean up the interface, improve the data structure, and decide what each user role needed to see. After that, several parts moved forward at the same time. The interface changed while the KPI logic was still being discussed. New features also required backend and database changes. The machine learning work started later, after much of the main app had already been built. Its purpose was to check whether public data supported the kinds of risk signals we wanted to show.

2. Project Implementation

2.1 Steps Taken to Execute the Project

We built one web application with separate areas for patients, trainers, clinicians, and administrators. These areas shared a backend and a relational database. The technical work mattered, but many of the hardest choices were about the content. We had to decide which data should be combined, what a result should say, and how to stop a score from sounding more certain than it was. We used generative AI tools to help with coding, debugging, and revisions. This saved time, but the team still had to define the requirements, test the output, connect the different parts, and correct problems.

The patient area changed the most during Project II. It began as a basic dashboard. It later included the WellPath, Activity, and Consistency scores, detailed metric cards, trend views, mood tracking, nutrition tracking, cycle tracking, AI explanations, clinical information, and support from trainers or clinicians. The trainer area focused on activity, recovery, goals, and notes. The clinician area focused on patient history, trends, review signals, and reports. The machine learning models and the AI explanations were kept

separate. The models tested patterns in public data. The AI was mainly used to explain the information in simpler words.

The first practical step was to turn the proposal roadmap into smaller parts that could be built. The team reviewed the planned data, the user roles, the KPI ideas, and the first dashboard. We then separated the work into the interface, database, API, AI, and research areas. This made the broad idea easier to manage, although the parts still depended on each other.

The integrated repository was set up with a React and Vite frontend, an Express backend, and a MySQL database [15]. Sample accounts and seed data gave each role something realistic to open. Environment templates were added so each team member could enter local database and login settings without putting those values in the source code. A seed command created the demonstration records, while a separate migration added patient preferences, connection records, and dated food logs.

The database grew beyond the first simple health log. It stored users and roles, patient profiles, daily health records, KPI values, goals, recommendations, care assignments, alerts, trainer notes, preferences, food logs, AI cache entries, and access history [15]. This change supported longer histories and role relationships. It also meant that a change to one field could affect several screens, routes, and seed files.

Backend work replaced much of the early browser-only state. Login used hashed passwords and returned a signed session token. Protected routes checked both the token and the role before returning patient, trainer, clinician, or administrator information. The frontend then called those routes for summaries, trends, notes, plans, food records, preferences, and AI requests. Local browser storage remained useful for a few display choices and for the small AI memory, but it was no longer the only place holding the demonstration state [15].

Feature work was added in stages. The first integrated version connected the main application and research folders. The next updates added the patient AI and partner pages, followed by larger trainer, clinician, administrator, nutrition, and personalization changes. Later updates corrected the nutrition history, brought the colours and typography into one theme, added responsive desktop behaviour, and let users change

metric goals. Building in stages made it possible to demonstrate progress, but it also caused older screens and newer data structures to overlap for a while [15].

The research work followed a separate path. Public NHANES and Add Health data were cleaned, divided for model development and testing, and used to compare several condition models [14]. The application did not train on the fictional users in MySQL. Results that were strong enough to explain were summarized in the Risk Signals and Data Insights areas. Keeping the research separate stopped a sample patient screen from being presented as direct medical evidence.

The patient Today screen was built as the main entry point. Score helpers combined selected daily measures into the WellPath, Activity, and Consistency summaries. The team then added detail cards so a user could see the values behind a score instead of receiving one unexplained number. Breakdown pages added 30-day, 90-day, and one-year views, goal progress, recent highs and lows, and simple links between measures. Goal values could later be adjusted by the patient, which made the prototype feel less fixed.

Mood and nutrition required a different kind of implementation because they added user-entered records. Mood entries had to be stored by date and compared with sleep, heart rate, and sitting time without claiming that one caused another. Food logging needed a dated history, a manual or AI estimate, and cautious pattern checks that only used days with matching records. The nutrition work also connected logged habits to the Risk Signals page so the two areas did not look unrelated.

Professional workflows were built around actions rather than extra dashboards. Trainers could choose an assigned patient, review recovery and recent activity, prepare a weekly plan, record a completed session, read patient feedback, and save encouragement. Clinicians could search patients, compare trend ranges, review a signal, write a secure note, prepare follow-up work, and export a summary. Administrators could review accounts, access rules, connections, and audit activity without opening the patient's private readings.

The AI work included both the request flow and the controls around it. Patient questions sent selected health context to the backend, where the Groq prompt added limits and

asked for short plain wording. The AI opt-out was checked by the interface and the API. A separate memory request looked for lasting details in typed messages, stored up to 20 short facts in the same browser, and included them with later questions. This allowed simple personalization without treating the chatbot as a medical record [15].

2.2 Task Breakdown and Team Roles

The team started with separate areas of responsibility, but more overlap was needed near the end. Yonathan worked on the early concept, planning, and client communication. J.J. led much of the visual design and interface work. Yeven designed and later updated the database. Buntong worked on backend logic, AI integration, and prompts. Mahir worked on the research and KPI direction, presentations, Project II planning, scope choices, integration, final changes, and the later machine learning work. These areas depended on each other, so team members often helped outside their main role during integration.

Although each member had a main area, the work could not stay completely separate. A new database field had to match the backend response and the frontend name. A new role screen also needed authentication, sample data, responsive styling, and a way to handle missing information. Team members therefore moved between their assigned areas during integration instead of waiting for one person to finish every dependency.

Handoffs worked best when they included a working example. The frontend handoff listed the screens, temporary state, planned routes, role boundaries, and safe AI wording. The database and backend work then replaced parts of that temporary setup. This approach helped the team connect a large interface, but some older instructions remained after the structure changed. The README still referred to a separate desktop wrapper after that wrapper had been removed, which showed that documentation needed to be updated at the same time as code [15].

Team roles also changed as the prototype became integrated. Interface work could not be considered finished until the matching data and route existed. Database work could not be checked only through SQL because a screen still had to read and save the value correctly. Research findings needed plain explanations before they could appear in the

application. This made integration and review shared work even when one member had written most of the first version.

The final handoff included setup files, seeded demonstration accounts, start and stop scripts, a presentation checklist, and notes about known limits. These materials reduced the chance that the demonstration would fail because someone forgot the database, API, or environment settings. They also recorded fallback steps for port conflicts, missing data, and unavailable internet during live AI use [15].

2.3 Challenges Faced and Mitigation Strategies

The hardest problem was keeping a clear direction while the project kept changing. The original idea was broad. During Project II, we had to decide how far to expand it without making the app look like a medical tool. Client communication was also limited during parts of the semester. This made it harder to confirm that every change still matched the client's expectations. We went back to the first meeting notes and the Project I proposal. We also used regular feedback from class and the instructor to guide the next steps.

Another challenge was the lack of medical review. We could read research and test public data, but we were not qualified to approve the app as a clinical tool. This became more important when we added scores, risk signals, and prediction models. We responded by using careful wording, keeping the research separate from the AI explanations, and removing or lowering confidence in weak model results. A major team issue and uneven availability also broke the original schedule. We moved tasks between members and completed more integration work at the same time. This made the final weeks much more crowded than planned.

Scope growth was the first challenge. Mood, nutrition, cycle tracking, professional workspaces, the admin area, research models, and AI personalization all had value, but they could not receive the same level of testing. We responded by keeping the core role paths working and treating native device connections, full survey storage, and production monitoring as future work. This kept the demonstration useful while making the remaining limits visible.

Data mismatches caused another group of problems. Frontend components and backend routes sometimes used different names or expected different record shapes. The food log was one clear example. A mismatch had to be corrected before dated meal history and nutrition patterns could use the same records. Similar issues appeared in role permissions and fallback data. The main response was to fix the shared field names, improve error states, and test the affected screen with the backend running [15].

Visual problems also increased as the app became responsive. Charts were cut off, light-mode labels disappeared, and desktop and phone rules sometimes overrode one another. Some controls looked active even though their content had not been built. We reduced these problems by using shared theme values, giving charts stable space, removing dead controls, and checking the four roles at several widths. The final design still needs a recorded cross-browser test, but the most visible failures were corrected.

AI required repeated adjustment rather than one prompt. Early Groq responses could be too general, too long, or too confident. The backend instructions were tightened so the answer had to use available WellPath data, avoid diagnosis and treatment, state when context was missing, and give a short lifestyle step. The memory function was also limited to short lasting facts instead of saving the whole chat. These restrictions made the feature more useful for a health project, but they did not remove the need for human review [13], [15].

Another challenge was deciding when a feature was complete. A page could look finished while still using sample fallback data or missing a save action. We began checking completion through a full user path. For example, a nutrition feature had to accept an entry, keep its date, show it in history, and use it in later pattern text. A clinician note had to remain attached to the selected patient. This reduced the chance of counting a visual mock-up as a working result.

Setup differences between team computers also caused delays. The integrated app needed Node, the correct package versions, MySQL, an imported schema, seed data, environment settings, and two running processes. Environment templates and launcher scripts made setup easier, but the project still depended on local machine conditions. A

hosted test environment would have reduced this problem, but it was outside the time and privacy scope of the course prototype.

3. Monitoring and Adjustments

3.1 Methods Used for Tracking Progress

We tracked progress with the original roadmap, weekly task planning, class feedback, Discord updates, and direct messages between team members [2]. We did not use a full Scrum process or keep a complete issue log. Instead, we worked in short planning periods. After feedback, we usually turned the main changes into tasks for the next one or two weeks. We also checked the remaining work against the semester schedule. If one delay affected another task, we changed the order or reduced the scope. This worked, but the process became more reactive after the schedule changed.

We also used working demonstrations, screenshots, and GitHub commits to track the interface. We reviewed each major version instead of waiting for the last week. This helped because the project grew from one patient dashboard into four role-based areas. GitHub gave us a record of the larger changes. Smaller layout choices were usually discussed by the team and then checked in the running app [15].

Our checks followed common tasks for each role. In the patient area, we opened cards, moved between the main sections, changed the theme and display settings, and checked that saved choices stayed after a refresh. We also selected clients and updated plans in the trainer area, reviewed signals and notes in the clinician area, and checked account and access screens in the admin area. These were useful manual checks, but they were not a complete automated test process.

The monitoring process used several kinds of evidence. The proposal and Week 4 presentation showed the planned direction [1], [2]. Git commits showed when larger code changes entered the integrated repository. Working demonstrations showed whether a user could finish a task. Screenshots helped compare the layout at different stages. The team also reviewed the database and API when a screen showed the wrong value or failed to save.

Role walkthroughs were repeated after larger updates. For a patient, the check covered login, the Today screen, detail pages, mood, nutrition, risk signals, goals, partner support, and saved settings. Trainer checks covered client selection, plan changes, session records, feedback, and encouragement. Clinician checks covered patient selection, trend ranges, signals, secure notes, follow-up plans, and export actions. Admin checks focused on accounts, permissions, connections, and audit activity without private readings.

Research results were monitored separately from interface progress. The model work checked sample size, missing fields, train and test separation, AUC, and whether an added group of variables improved the result. This was important when wider social and background factors were tested. Some sounded relevant but did not improve the models. The team therefore kept those findings cautious instead of treating every available field as useful.

3.2 Significant Changes Made During the Project

The biggest change was the wider scope. By the middle of Project II, the team felt that the original dashboard and simple rule-based insights did not show enough of the idea. Feedback also pushed us to look again at the wider health context. We added higher-level scores, revised KPI views, AI explanations, mood and nutrition tracking, cycle tracking, stronger professional tools, and a machine learning research section. These changes came in stages, but each one created more work for the interface, backend, database, and testing.

Testing at phone, tablet, and desktop widths exposed problems that screenshots did not show. Some charts were cut off. Light-mode labels were hard to read. A few clinician tabs looked selected without changing the content, and some API errors replaced an entire page when the backend was not running. We also found animations and dropdowns that moved too quickly.

We fixed the clearest problems by giving cards and charts stable sizes, moving colours into shared theme settings, improving loading and error states, and checking every role at more than one width. The Git history shows two major role and personalization updates on August 8, a nutrition and theme update on August 15, and a responsive

desktop update on August 16. The last update also removed the separate web wrapper and added automatic, app, and desktop display choices [15]. This gives a clearer record of how the interface changed, but it also explains why testing was pushed near the end.

The schedule also changed. The first roadmap showed a seven-week sequence for data preparation, KPI logic, integration, interface work, explanations, testing, and the final presentation. The work did not happen in that order. Interface changes affected the data we needed. New features required database changes. KPI work continued after visual feedback, and the machine learning work started later than planned. Uneven availability created more delays. Changing the order helped us finish a stronger prototype, but it left less time for formal testing and final client review.

A major adjustment came from the difference between a polished screen and a complete workflow. Early work gave the patient area the most attention. Later reviews showed that the trainer and clinician roles needed actions of their own, not just access to patient information. The August 8 updates expanded those areas with plans, sessions, notes, follow-up work, reports, and clearer role boundaries [15].

The August 15 update responded to problems found in nutrition and visual consistency. Food records gained dates and history, a schema mismatch was corrected, and nutrition explanations were tied to logged foods. Settings moved to a clearer location, the colours were brought into one teal-based theme, font weights were cleaned up, and a duplicated AI page was removed. These changes made the product easier to follow but also required another round of checking [15].

The August 16 to August 20 updates focused on responsive layout, simpler navigation, risk-signal profile details, adjustable metric goals, and score interactions. The separate desktop wrapper was removed in favour of one application with Auto, App, and Desktop choices. This reduced duplicate work. It also meant the integrated app had to support more screen sizes at once [15].

The late social-determinants revision changed both the research and the wording of the project. The health survey preview added some background questions, but it did not save them. The model work then tested whether wider background and lifestyle data

improved the results. AUC and confidence labels were added because a risk score without a measure of model performance could give the wrong impression. This was a meaningful correction, although it arrived too late for full integration.

4. Results and Analysis

4.1 Data and Evidence of Project Outcomes

The final WellPath prototype went beyond the first Project I design while keeping the same main purpose. The patient dashboard brings daily wellness information into one screen. It begins with the WellPath Score, Activity Score, and Consistency Score. Users can then open details for sleep, steps, recovery, resting heart rate, nutrition, and other information. The goal is to help a user notice a change, compare it with their usual pattern, and decide whether it may be worth reviewing.

The other user roles show the same data in different ways. Trainers see information about activity, recovery, goals, and support. Clinicians see longer health histories, intake information, trends, notes, follow-up items, and reports. The admin area supports account and system tasks but does not give broad access to private health details. Figures 1 and 2 show the patient and clinician views. Figures 3 to 6 show patient features that were added during Project II.

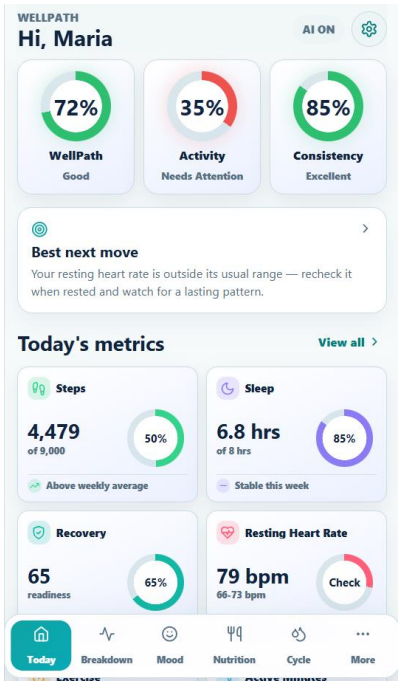

Figure 1. Patient dashboard showing the three main scores and the daily metric cards.

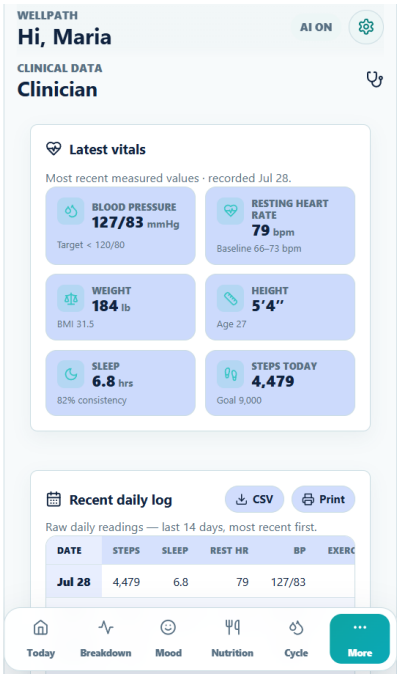

Figure 2. Clinician view showing the latest vital signs and recent daily records.

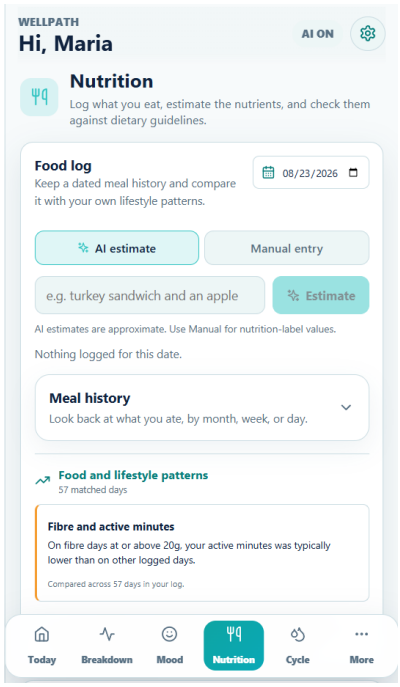

Figure 3. Nutrition tracker showing the food log and the optional AI estimate tool.

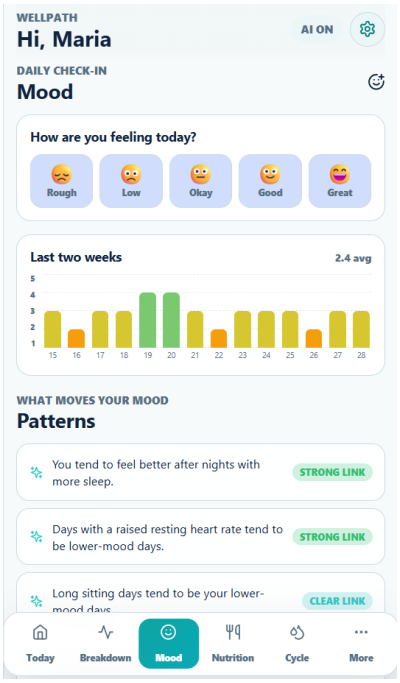

Figure 4. Mood tracker showing the daily check-in and patterns from recent entries.

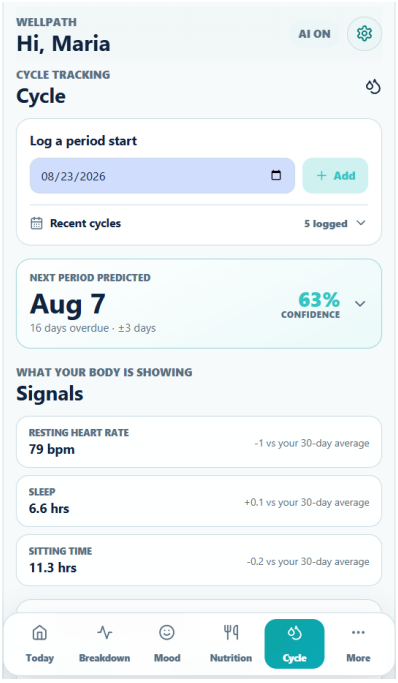

Figure 5. Cycle tracker showing the next estimated period date based on the sample patient's past entries.

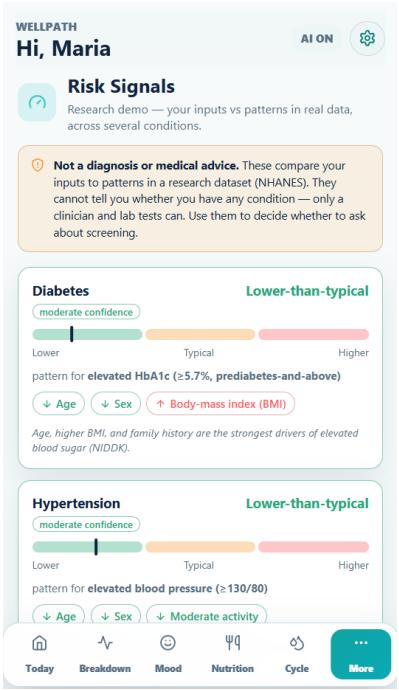

Figure 6. Risk Signals screen showing lower-than-typical, typical, and higher-than-typical bands. These bands are not diagnoses.

Figure 5 shows the cycle tracker. It estimates the next period date from the dates entered in the sample patient's history. The confidence percentage reflects how regular that history is. It is not a medical test or a guarantee that the date will be correct.

Figure 6 shows the Risk Signals screen. Each band compares the sample patient's values with patterns found in public data. The confidence label refers to the strength of the model behind the signal. It does not mean that the patient has that condition. The model testing is explained in the next part of this section.

The patient area includes more than the main cards. The Breakdown screen lets a user compare 30 days, 90 days, or one year, look for links between different habits, and adjust a goal when the current target is not realistic. The mood page compares check-ins with sleep, sitting time, and resting heart rate. The nutrition page keeps a dated food log, offers an optional AI estimate, and connects eating habits to the Risk Signals screen. The support page lets a patient manage goals, send a post-session check-in, choose what a trainer can see, and review a simple partner plan [15].

The professional workspaces also include tasks that are not clear from the screenshots. Trainers can build weekly plans, log completed sessions, review patient feedback, and write encouragement notes. Clinicians can search assigned patients, compare 7, 14, or 30 day trends, save private notes, manage follow-up plans, and export a plain-language summary. Administrators can review account status, role access, connection status, and audit activity without opening patient readings or private notes [15].

To test the research side of the idea, we used two cycles of the National Health and Nutrition Examination Survey, or NHANES. NHANES is a large U.S. survey that includes interviews, physical exams, and lab tests. We combined the 2015-2016 and 2017-2018 cycles and had data from 11,848 adults [3].

We split the people into two groups. The model learned from the first group, called the training set. We checked it on the second group, called the held-out test set. Held-out means the model had not seen those people during training. This gave us a fairer test of how the model handled new data. We also compared each full model with a basic model that used only age and BMI.

We reported performance with a score called AUC [5]. AUC shows how well a model separates people with and without the result being studied. A score of 0.5 is about the same as random guessing. A score closer to 1.0 is better. We also reported a 95 percent confidence interval. This is a range that shows how much the AUC changed when we repeated the test with many resampled versions of the test data [7]. A narrow range gives us more confidence that the result is stable.

The strongest result was elevated HbA1c, with an AUC of 0.814 and a confidence interval from 0.795 to 0.832. HbA1c is a blood test that reflects average blood sugar over the past two to three months. Elevated blood pressure reached 0.740. The kidney UACR model reached 0.708. UACR is a urine test that can point to kidney stress. Low HDL, sometimes called good cholesterol, reached 0.694. High triglycerides reached 0.651, while high LDL reached only 0.572. We kept the stronger models and marked or removed the weaker ones instead of treating all results as equally useful.

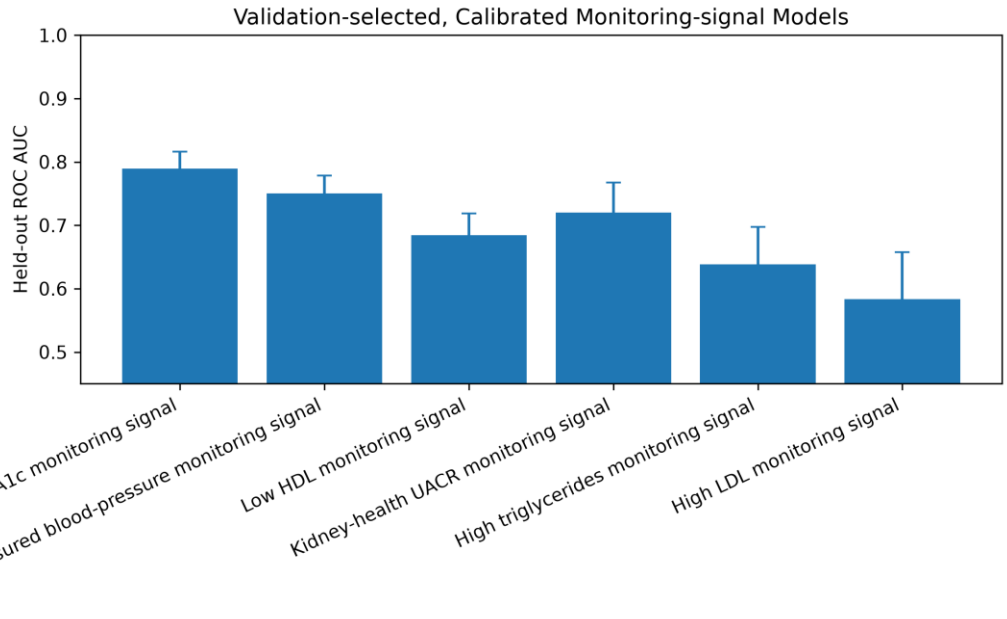

Figure 7. Test results for six risk-signal models. Higher bars mean better separation. The thin lines show how much the result changed across repeated tests.

Figure 8 compares several model types for the HbA1c result. The ROC curve shows the tradeoff between correctly finding higher-risk cases and creating false alarms at different settings [5]. The gray diagonal line shows random guessing. A curve that stays farther above that line is doing a better job.

We tested logistic regression, a decision tree, a random forest, gradient boosting, and a simple point-based risk score using scikit-learn [6]. Logistic regression, random forest, and gradient boosting were close, with AUC scores around 0.80 to 0.81. The decision tree and point-based score were lower. Since the top scores were very close, we did not choose a model only because it had the largest number. We also wanted a result that could be explained and adjusted so its probability scores better matched the rates seen in the data. That adjustment is called calibration.

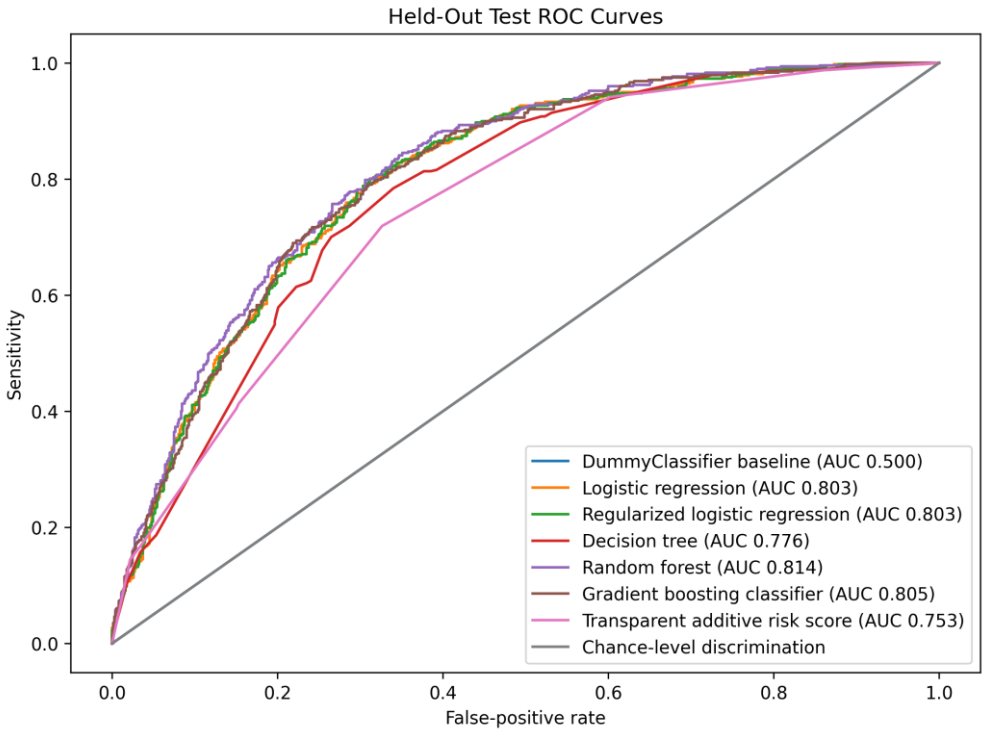

Figure 8. ROC curves for the HbA1c models. Curves farther above the gray diagonal line show better separation. The AUC values in the legend summarize each model.

The models that performed well enough were connected to the app's sample patient data. On the Risk Signals screen, the sample patient's values are compared with patterns learned from NHANES. The app then shows a lower-than-typical, typical, or higher-than-typical band. The confidence label describes how strong the supporting model result was. It does not show the chance that the patient has a disease, and it is not a diagnosis. The research scripts and result files are kept with the project materials [14].

The Add Health study gave us a way to look at the same people over many years [4]. Our first analysis used 1,760 people who had lab data in both 2008 and 2016. Adding the earlier blood tests raised the AUC for later high blood pressure from 0.640 to 0.667. The change was small, but it showed that earlier blood sugar and triglyceride results added some useful information.

A second Add Health analysis linked 4,304 teenagers to their own adult HbA1c results about 14 years later. A model using only age and sex had an AUC of 0.565. Adding the teenage health and background profile raised it to 0.618. Teenage BMI was one of the

clearest factors. The adult elevated-HbA1c rate was 41 percent in the heaviest third of teenagers and 24 percent in the leanest third.

This later work also included wider social and background factors, such as parent education. Parent education showed a small link with lower risk, but adding more social, nutrition, and background data did not always improve the models. These results did not prove cause and effect or patient benefit. They showed us that a factor should not be added to a risk signal only because it sounds relevant. It still has to improve the result in the data.

4.2 Analysis and Alignment with Objectives

When we compared the final prototype with the Project I goals, some goals were clearly met. The app now brings patient information into one system, shows changes over time, and gives each user role a different view. The patient dashboard, clinician view, trainer tools, and admin area all support this goal. We also moved from static screens to stored data and working account roles.

Other goals were not handled well at first. Social determinants of health were one example. This term means non-medical conditions that can affect health, such as income, education, housing, food access, neighbourhood, and social support. Our early prototype focused much more on health readings, activity, sleep, and the interface. We did not give enough attention to these wider factors until later feedback made us return to that part of the project.

The later health survey preview was one way we tried to correct this gap. It begins with consent and then covers body measures, family history, lifestyle, movement, and the user's main goal. Optional questions add some wider context, including ethnic background, the first three characters of a postal code, and whether a normal workday is mostly sitting or active. Sensitive questions explain why they are being asked, and the survey can calculate a simple summary on screen. However, it is still a front-end intake prototype. Its answers are not yet saved to the main database or used across the risk models. It also does not fully cover income, education, housing, food access, or social

support. This is why the wider social context is still described as only partly implemented [15].

That revision led to the later research work. We tested whether lifestyle, social, and background data actually improved the models. We also added AUC results so we could show how well each model worked instead of only showing a risk score. The Risk Signals page came after this work. It was added so the screen could be tied to tested public data and could show uncertainty more clearly.

The final project did not meet every objective in a straight line. Role-based views and trend tracking were built earlier and improved over time. Social factors and tested risk signals came much later and are only partly built into the app. The models have also not been approved for clinical use. The project met its main prototype goals, but the wider health context and real-world testing still need more work.

The frontend objectives were met at different levels. The patient summary, separate role areas, and main trainer and clinician workflows all work in the demonstration. The admin area and phone and desktop layouts are also present. However, the newest nutrition, theme, navigation, and responsive changes were made close to the report deadline. They still needed one final check across all four roles. For that reason, we can say the workflows were demonstrated, but not fully validated [15].

5. Risk Management

Several risks from Project I continued into Project II. Privacy and access control were important because the app handles sensitive information for different user roles. The scope also grew much more than planned. The schedule became a real risk after a major team issue and uneven availability changed the order of work. The largest content risk was making a score or AI explanation sound more certain than the research supported.

We managed these risks by limiting what the prototype could claim. The app used made-up patient data, and the research used public datasets. Health results were described as wellness information rather than a diagnosis. User roles were separated so people only saw the information needed for their work. Lower-priority production

features were delayed when the scope grew. For the models, we tested results on data that the models had not seen before and checked for data leakage. Data leakage happens when information from the test data accidentally helps train the model. The risks we handled least well were outside review and client agreement. We had limited client feedback near the end and no clinician available to approve the health content.

Role boundaries were one practical way we reduced privacy risk. Trainers can work with assigned activity, goals, sessions, and feedback, but do not need private mood entries, clinician notes, or AI conversations. The admin area handles accounts, roles, consent records, connections, and audit events without showing individual readings. These limits are built into the interface, but they still need stronger automated authorization testing [15].

The Project I proposal grouped its risks into technical, security, project, analytics, and ethical areas [1]. All five appeared during Project II. The difference was that they became easier to see once the system had working accounts, stored data, AI responses, and research results.

The main technical risk was integration. The interface, API, database, and research code were developed at different speeds. Field-name differences, missing backend services, seed data, and environment settings could stop a screen from working. We reduced this risk with sample accounts, environment templates, seed scripts, migrations, fallback states, and repeated role walkthroughs. The impact was not removed. The final build still depends on local setup and does not have one automated command that tests the whole system [15].

Security risk centred on role access and sensitive information. Server routes checked login tokens and roles, while the interface kept trainer, clinician, and administrator tasks separate. Made-up patient records avoided exposing real health data. These choices lowered the risk during development. They do not replace a full security review, stronger consent controls, or automated tests that try to access another role's data.

Project risk came from scope and time. The number of features grew while the team schedule became less stable. The response was to move work between members,

combine some tasks, and leave native health-device testing and production features outside the final scope. This allowed a broader prototype to be completed, but it pushed formal testing and documentation toward the deadline.

Analytics and ethical risks were closely connected. A model result, risk band, or AI answer could be mistaken for a diagnosis. We used held-out model testing, AUC, confidence wording, non-diagnostic labels, AI restrictions, and clear limits throughout the report. The remaining risk is still important because no clinician reviewed the final content and no real users tested how they understood the language. A future pilot would need medical, privacy, accessibility, and fairness review before any stronger claim could be made.

6. Economic and Business Value

6.1 Estimated versus Actual Value

The Project I proposal described WellPath's value mostly through the expected benefits for each user. Patients could understand their daily habits more clearly. Clinicians could see more information from between appointments. Trainers could support activity and recovery. An organization could later study larger patterns using grouped data. The final prototype shows these ideas on a small academic scale. It does not prove that the same benefits would happen in a real clinic.

Project I described expected value by user rather than giving a dollar forecast [1]. Patients were expected to gain a clearer view of daily habits. Clinicians were expected to see lifestyle information that is normally missing between visits. Trainers were expected to use recovery and engagement information. Organizations were expected to learn from larger grouped patterns. Because the proposal did not set a starting budget, revenue target, or expected number of users, the final comparison has to focus on delivered value and remaining evidence gaps.

The prototype delivered part of that expected value. Each role now has a separate workflow, and the patient view combines more information than the first mock-up. The system also shows how preferences, notes, plans, food logs, trends, and risk signals could be stored and reviewed together. What it did not deliver was proof that these

features save time, change behaviour, improve health, or reduce cost. Those outcomes require real users and an agreed pilot measure.

One unexpected value was the research result itself. The team learned that adding more social, lifestyle, and background fields did not always improve a model. That finding prevented the report from claiming that a larger data collection plan would automatically create better insight. It also suggests that a real organization should test each added field against a clear purpose instead of collecting sensitive information just because it may be available.

6.2 Cost-Benefit Analysis

The direct software cost was low because we used open-source tools, small development services, made-up patient data, and public research datasets. The main cost was the time spent by the team. We did not keep a complete record of work hours, so we cannot give an exact labour cost. A real product would also need paid hosting, system monitoring, security testing, privacy work, data connections, technical support, and medical review if stronger health claims were made.

The benefits are also hard to measure because the app was not used by real patients or health organizations. We can show that the prototype works, presents health information more clearly, and gives each role a useful workflow. We cannot yet show saved clinical time, better health results, higher user activity, or a financial return. The fair conclusion is that WellPath has possible business and workflow value, but a real cost-benefit study would need a pilot project with clear measures.

The project avoided several direct costs by using open-source development tools, fictional patient records, and public research datasets. One responsive application also replaced the idea of maintaining separate phone and desktop projects. These choices lowered software and duplication costs for the course project. They did not remove labour cost, which was the largest actual input.

The team did not keep a shared record of hours, so an exact labour cost would be invented. The work included interface design, database changes, backend routes, AI prompt testing, model development, integration, presentations, and final checking. A

better project process would record hours by work area from the start. That would allow the final report to compare planned effort with actual effort and show where scope growth used the most time.

A real version would add costs that the course prototype did not face. These include secure hosting, backups, monitoring, penetration testing, legal and privacy review, accessibility testing, device connections, support, medical review, and ongoing model checks. Groq use would also need a clear service budget and data-handling agreement. If the product moved into a clinic, connection work with existing systems would likely be a major cost.

The next cost-benefit step should be a small pilot with measures agreed in advance. Useful measures could include how long a user needs to find a trend, how often a trainer updates a plan, whether a clinician finds the summary useful, how many users finish the survey, and how often an AI explanation must be corrected. Those results could be compared with setup, support, and review time. Until then, the project shows possible value and technical feasibility, not a financial return.

7. Engineering Design Outcomes

7.1 Final Design and Modifications

The final design is one web system with different views for patients, trainers, clinicians, and administrators. A shared backend and database keep the screens connected to stored records. The front end uses React and Vite [9], [10]. An Express API handles login and role-based routes [11]. MySQL stores users, roles, daily health records, preferences, notes, and demonstration data [12]. Passwords are hashed with bcrypt. Login sessions use JSON Web Tokens, or JWTs [8]. Server checks also confirm the user's role before protected data is returned. Optional AI explanations use Groq and can be turned off [13]. The machine learning research runs in a separate Python process and does not train on the app's demonstration database. This separation matters because the models create the signal and the AI only helps explain it.

Groq was used for the short explanations shown at the bottom of several patient pages [13]. Early replies were often too general or sounded too certain, so the prompts were

revised many times. The final prompts tell the AI to answer the user's question directly, use only the available WellPath data, keep the wording short, and give one realistic lifestyle step. They also stop it from diagnosing a condition, suggesting treatment, prescribing medication, inventing missing values, or presenting a weak pattern as certain. If important data is missing, the response is supposed to say that instead of guessing [15].

The chatbox also has a small personalization memory. When a user types a lasting detail, such as working night shifts, having knee pain, following a vegetarian diet, or training for a goal, the backend turns it into a short fact. The browser stores up to 20 of these facts for that patient and includes them with later AI requests. This helps the AI avoid advice that does not fit the person's routine or limits. It does not save the whole conversation, and the saved facts stay on the same browser and device [15].

Several design choices changed while we built the project. The database was expanded to hold longer histories, care relationships, scores, alerts, goals, recommendations, and notes. The patient screens were reorganized more than once so the main results were easier to find. We also moved away from treating AI as the main feature. The final design puts more focus on calculations, trends, and tested models. AI remains an optional explanation tool.

The interface layout also changes by role. Patient and trainer tasks are short and frequent, so those areas use phone-style layouts with compact cards and quick actions. Clinicians and administrators need to compare records and work through longer lists, so those areas use wider layouts. We kept these views inside one responsive application instead of maintaining separate mobile and web projects. This reduced duplicated screens and helped keep the features in sync [15].

The patient area also gives users some control over presentation. A user can choose which cards appear, switch between light and dark themes, adjust spacing and animation, choose automatic, app, or desktop view, and turn AI explanations off. These settings change how information is shown, not the health values. We also improved visible keyboard focus, contrast, touch control sizes, and reduced-motion support.

These changes improved accessibility, but we did not complete a formal accessibility audit [15].

Figure 9. Data Insights screen showing research findings inside the patient area.

In a normal request, the browser first sends login details to the Express API. The API checks the password, returns a signed token, and uses that token on later requests. The route then checks the user's role before reading or changing MySQL records. The response goes back to the React screen, which turns the data into cards, histories, plans, or notes. This flow keeps access decisions on the server instead of trusting a hidden button in the interface [15].

Calculated scores, research signals, and Groq explanations also have different jobs. Score helpers use stored or sample measurements. The research project tests patterns in public data. Groq receives selected results and context only after those steps, then writes a short explanation. This order makes it easier to say where a result came from and prevents the AI from becoming the source of the health signal.

7.2 Design Decisions and Justifications

Our best design choices made it easier to see where an insight came from. Separate workspaces kept each role focused and reduced access to information they did not

need. Records over time made trends more useful than one reading. Scores gave users a quick summary, while the detail pages showed how each score was built. We also kept model results separate from AI-written explanations. This stopped the app from making it look as if a chatbot had discovered a medical pattern on its own.

We also kept data requests separate from the visual components. This let us build screens with sample information first and connect them to the API later. It also made integration problems easier to locate. Several errors still came from the frontend and backend using different field names or expecting different permissions. This showed why shared data rules need to be agreed on early [15].

7.3 Design Limitations

The final design still has clear limits. It has not been used by a health organization or reviewed by a clinician. It does not connect to real wearables or clinical systems. Apple Health and Health Connect appear only as planned connection points and were not tested with physical devices. Some parts of the app and the research are still separate. The app also depends on a local MySQL database, environment settings, sample accounts, and separate frontend and backend processes. These limits are acceptable for a course prototype, but not for a released health system [15].

The personalization memory has its own limits. It stays in local browser storage, so it does not follow a user to another device or browser. The current interface also does not provide a full screen where the user can review and remove each remembered fact. A real version would need clearer consent, stronger storage, and direct controls over what the AI remembers [15].

Testing is another limit. The repository has commands to build the frontend and start the backend, but it does not have one automated test command for the integrated app. Most checks were manual. Recent changes to nutrition, themes, navigation, and responsive layout need a final check across all roles. This kind of repeat check is often called regression testing. It means making sure a new change did not break something that worked before. Until those results are recorded, the final build should not be described as fully tested [15].

8. Timeline Review

The Project I roadmap showed a seven-week plan. It started with sample data and the database. It then moved through KPI rules, dashboard connections, interface work, explanations, testing, and the final presentation. Project II did not follow that order. Early work focused on interface cleanup, mobile layout, database changes, intake fields, and KPI planning. Later work was meant to cover AI, security, testing, final polish, and handoff. In practice, these tasks overlapped and often changed order.

There were a few main reasons for the change. A major team issue made the original schedule harder to follow. New features also created more links between the interface, backend, database, and research work. KPI and score designs continued to change after feedback. The machine learning work became a separate late project after most group development was complete. Client contact was also limited near the end. The revised schedule gave us a wider and more polished prototype, but testing and documentation were pushed into the final weeks. In a future project, we would keep extra time for integration and unexpected team problems.

The proposed roadmap started with data preparation, then moved to database design, KPI logic, dashboard development, interface refinement, explanations, and testing [1]. In the actual project, these stages overlapped. The database changed after interface work began, KPI rules changed after visual feedback, and research started after most application features were already in place.

The integrated repository first appeared on August 3 with the application, models, and research folders. Environment templates were added on the same day. On August 4, patient AI, partner support, and personalized insight work were added. These changes moved explanation features earlier than the original roadmap expected [15].

The two August 8 updates expanded the trainer and clinician workspaces, authentication flow, responsive styling, patient settings, nutrition, administrator tools, and backend connections. This was the point where the project clearly changed from a patient dashboard into a four-role system. It also increased the number of workflows that needed to be tested [15].

From August 15 to August 20, the team corrected nutrition history, standardized the theme, removed duplicate navigation, added desktop display choices, tightened risk-signal profile details, added adjustable goals, and polished score interactions. These were useful final changes, but they happened during the time that had originally been reserved for testing and finalization [15].

The actual schedule produced a wider prototype than the original plan, but not the same level of completion in every area. Role views, trends, and AI explanations became visible strengths. Survey storage, direct research integration, native device testing, and automated end-to-end checks remained incomplete. A future timeline should include an early integration checkpoint, a feature freeze, and at least one full week where no new feature is added.

The roadmap also lacked clear exit checks between stages. Data preparation was not truly finished before dashboard work began, and interface refinement continued while new features were still being added. This made the last testing stage absorb work from almost every earlier stage. In a future schedule, each stage would end with a small result that can be checked, such as an approved field list, a working route, or a completed role walkthrough.

There were no dependable early completions to report because most tasks fed into later integration. Some individual screens were ready early, but they changed again after the database, role permissions, or responsive layout moved forward. The more useful comparison is that explanation and role features arrived earlier than planned, while formal testing, survey storage, device connections, and final documentation finished late or remained incomplete.

9. Lessons Learned

One of the biggest lessons was that a broad idea needs more than a long feature list. At first, we assumed that adding more health data would create better insights. The research showed that this was not always true. Some factors helped the models, while others added almost nothing. We learned that it is better to start with a clear question, test the evidence, and then decide whether a feature should be built.

We also learned about project management and the limits of AI tools. Discord gave us one place to keep updates, but team members did not use it in the same way. More scheduled work sessions would have helped when one task depended on another. We also learned that delegation is not only about dividing tasks. The links between tasks have to be planned too. Generative AI helped with coding and debugging, but it did not make the main decisions for us. We still had to define what we wanted, check the generated work, connect it to the rest of the app, and fix errors. This was especially important in a health project. We had to know when a result was too uncertain and when the app should not make a stronger claim.

The frontend work added another lesson. A screen can look finished and still fail during ordinary use. The role walkthroughs found problems with spacing, navigation, charts, contrast, loading states, and access boundaries. We also saw how quickly integration can break when the frontend and backend use different field names or permissions. In a future project, we would agree on those data rules earlier, use smaller updates, and require a short check of each role before changes are added to the main branch.

Another lesson was that a handoff document can become outdated quickly. The early frontend guide was useful because it listed temporary state, expected routes, and role boundaries. Once the backend and responsive layout changed, some of those notes no longer matched the repository. Future handoffs should be updated in the same pull request as the code they describe.

The project also showed that evidence changes design. Social determinants sounded important from the beginning, but they were not built into the first versions. Later testing showed that some wider factors added little to the models. That did not make them unimportant to health. It meant the team had to separate social importance from model usefulness and explain both honestly.

Finally, health wording needs its own review step. A sentence can be technically careful and still sound like a medical conclusion to a user. The repeated changes to risk labels and AI prompts showed why a future project should include a clinician and ordinary users earlier. Their feedback should be recorded as part of testing, not left until the final presentation.

We also learned to define completion in terms of evidence. A feature is not complete because it appears in a screenshot. It should have a clear data source, a working action, an expected result, and a recorded check. Applying that rule earlier would have made the final testing smaller and would have exposed temporary fallback data sooner.

The business relationship side of the project needed more structure as well. When client contact became limited, the team returned to earlier notes and made reasonable choices. That kept work moving, but it did not replace approval. Future projects should record decisions, open questions, and the date of each client response. If contact stops, the team can then show which choices were confirmed and which were assumptions.

Some working habits should be kept. Demonstrating the application throughout the term made problems visible before the final presentation. Keeping fictional app records separate from public research data also prevented the team from overstating what the sample users proved. The role-based design gave each member a clear area to discuss, and the later Git history provided a useful record of major changes. These choices gave the project a stronger base even when the schedule became crowded.

The main improvement for a future project would be a shorter shared checklist that is used every week. It should record the owner, expected result, data source, role permission, test status, and any open question for each feature. Large updates should be divided into smaller changes that can be reviewed before they are combined. Client questions and medical wording questions should also have named owners. This would make the work easier to track and reduce the amount of integration left for the final weeks.

10. Conclusion

WellPath began with a simple Project I idea. We wanted to bring clinical, wearable, and lifestyle information into one place and make it easier to understand. Building the prototype showed that the idea was more complicated than it first appeared. We had to deal with scope, data quality, privacy, different user roles, clear explanations, team coordination, and the strength of the evidence. The final app is both a working prototype and a test of what a useful wellness platform could look like.

Within that scope, the project was successful. The final prototype is wider and more complete than the Project I version. It uses scores, history, patterns, research-based risk signals, and optional AI explanations to make health data easier to review. The project also showed its limits. We did not have enough client feedback at the end, we could not get medical review, and the research showed that more data does not always improve a prediction. These limits helped us understand what a real product would need. WellPath also followed important business systems practices. We gathered needs from different users, separated the front end, API, and database, limited access by role, and recorded where the results were still uncertain.

The project met the main Project I goal of showing how clinical, wearable, and lifestyle information could be brought into one role-based experience [1]. It also went further by adding stored preferences, professional workflows, research signals, AI restrictions, and clearer limits. The strongest result is not one score or model. It is the working separation between evidence, demonstration data, and explanation.

The next stage should be smaller and more focused. It should connect one approved data source, save the health survey with clear consent, run the research model through one controlled backend service, and test one patient and clinician workflow with real reviewers. That would give the team better evidence about usefulness without repeating the scope growth of Project II.

References

[1] WellPath Team, “WellPath Project I Proposal and Project Outline,” CST3116 Project I, Algonquin College, 2026.

[2] WellPath Team, “WellPath Project II Week 4 Presentation,” CST3117 Project II, Algonquin College, 2026.

[3] U.S. Centers for Disease Control and Prevention, National Center for Health Statistics, “National Health and Nutrition Examination Survey (NHANES), 2015–2016 and 2017–2018.” https://www.cdc.gov/nchs/nhanes/

[4] Carolina Population Center, University of North Carolina at Chapel Hill, “National Longitudinal Study of Adolescent to Adult Health (Add Health), Public-Use Data.” https://addhealth.cpc.unc.edu/data/

[5] Google for Developers, “Classification: ROC and AUC,” *Machine Learning Crash Course*. https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc

[6] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html

[7] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman & Hall/CRC, 1993.

[8] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, May 2015. https://www.rfc-editor.org/info/rfc7519/

[9] Meta Open Source, “React Documentation.” https://react.dev/

[10] Vite, “Vite Guide.” https://vite.dev/guide/

[11] OpenJS Foundation, “Express Documentation.” https://expressjs.com/

[12] Oracle, “MySQL Documentation.” https://dev.mysql.com/doc/

[13] Groq, “Groq Documentation.” https://console.groq.com/docs/

[14] WellPath Team, “WellPath Research Scripts and Analysis Outputs: NHANES Pooled and Add Health Analyses,” CST3117 Project II, Algonquin College, 2026. Internal project files.

[15] WellPath Team, “WellPath Integrated Application Source Code and Git Commit History,” CST3117 Project II, Algonquin College, 2026. Internal project repository.

Appendix A: Demonstration Accounts

The WellPath prototype uses fictional demonstration accounts for each user role. These accounts were created only to show how the different parts of the application work and do not represent real patients or health professionals.

The demonstration includes a patient account, trainer account, clinician account, and administrator account. Each account contains sample information so that features such as health trends, goals, notes, recommendations, and role-specific views can be demonstrated consistently.

Using fictional accounts allowed the team to test and present the system without collecting or storing real personal health information. In a real implementation, account creation, consent, privacy, and access procedures would need to follow the policies of the organization using the system.

Appendix B: Technical Setup and Verification Notes

The final WellPath prototype was tested mainly through repeated walkthroughs of the main user features. The team used the demonstration accounts to check that each role could access the correct areas of the application and complete its main tasks.

Patient testing included reviewing daily health information, opening trend and score pages, using mood and nutrition features, changing preferences, and viewing the available insights. Trainer testing focused on reviewing assigned patients, updating plans, recording sessions, and providing feedback. Clinician testing included reviewing patient information, trends, signals, notes, and follow-up items. The administrator area was checked to make sure account and system information could be managed without giving unnecessary access to private health details.

The team also checked the application during development rather than waiting until the end. Problems found during these walkthroughs included layout issues, inconsistent data between screens, incomplete actions, and differences between the mobile and desktop views. These issues were corrected as the project progressed.

The final prototype still has limitations. Some planned integrations, such as direct wearable connections and full survey storage, were not completed. Testing was also primarily manual rather than a full automated testing process. Because WellPath is an academic prototype, these limitations are documented rather than presented as completed production features.